

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors (urunc-dev/urunc#112)

Mentors: @cmainas,
@ananos

1. About me

I am Raman Kapoor, a B.Tech (Information Technology) graduate from IIIT Gwalior (class of 2026), currently working at Juspay, India's largest payments orchestration platform, on low-level Linux infrastructure. My most recent landed piece there was leading the migration of our block-storage tier from CEPH to Lightbits NVMe/TCP — same workload at 48K TPS versus 37K TPS on roughly a tenth of the hardware (four nodes instead of forty). Before that I shipped a thread-local and server-local cache layer in Rust on our hot APIs that cut cross-node calls by ~4% at peak, and a Go-based Shamir-secrets distribution broker on top of OpenBao that removed our reliance on AWS KMS for key delivery. Right now I am on the bare-metal datacenter side of the stack, writing the secure-erase command for our internal Rust CLI that drives MAAS-managed hosts through their lifecycle.

2. What I have been working on

2.1 Open-source contributions to Namma Yatri (Haskell, production)

The work I am proudest of in a large unfamiliar codebase with maintainer review is my contribution to Namma Yatri, the open-source ride-hailing platform that runs in production across India. Between November 2025 and January 2026 I shipped three merged PRs into the org:

- **feat** [nammayatri/nammayatri#13442](#) — Special-zone driver-payout scheduling system. A distributed, event-driven scheduler serving roughly 10,000 rides per day; cut driver payout latency from weekly cycles down to T+2 hours under production traffic.
- **feat** [nammayatri/nammayatri#13452](#) — Exposed the resulting driver-payout state to the driver-app frontend through the API layer.
- **feat** [nammayatri/shared-kernel#1108](#) — PaytmEDC payment-flow integration in the shared kernel: config management, checksum generation, API interaction. Went through several review iterations before merge.

The technically interesting one is #13442. Driver payouts for special-zone rides (airports, stations) are computed per-ride but were previously paid out only on a weekly batch cycle — a long latency for drivers, and operationally fragile when the batch failed. The new system schedules per-ride payouts at T+2 hours after ride completion, which means roughly 10,000 scheduled jobs in flight at any given moment under load. The constraints made the design non-trivial:

- **Exactly-once execution.** A payout that fires twice is real money lost; a payout that never fires is a real driver not paid. The scheduler had to be exactly-once under broker restart, app restart, and partial-failure scenarios.

- **No latency on the ride-booking hot path.** Scheduling a payout is a side-effect of completing a ride; the ride-completion API could not block on scheduling state.
- **Idempotent state transitions.** A retry of the payout-fire call had to be safe; the same input must yield the same end state, not double-debit.

The shape I landed on is a Redis-backed delayed-job scheduler keyed on ride-id, with per-ride distributed locks taken at fire time and an idempotency token persisted before the actual payout call. The per-ride lock ensures only one worker fires a given payout even if two pick it up simultaneously, and the idempotency token means a retry after a partial failure is safe. Working on a Haskell codebase with no prior production Haskell experience, designing the failure model with the team, and getting to merge three times in a row, is the closest thing I have done to what this LFX project will involve — and it is the muscle I am leaning on for that reason.

2.2 Juspay — production infrastructure work

2.2.1 Identity and secrets on OpenStack + OpenBao

We replaced a heavy AWS KMS dependency with an in-house identity-and-secrets layer running on OpenStack VMs and OpenBao. A few design choices shaped most of what I built:

- **Identity derived from the network, not from a long-lived token.** A VM authenticating to the broker presents an IMDS-attested claim — its identity is the network position the orchestrator placed it at, signed by the platform. There is no static credential on disk an attacker can steal. The broker validates the attestation and issues a short-lived RS256-signed token scoped to what that VM is allowed to read.
- **Shamir secret-sharing for the root key.** No single broker node holds the full root; it is split into m -of- n shards distributed across the cluster, and reassembled in memory only when needed. A single compromised node yields no useful secret material.
- **HA via Raft.** Brokers run a Raft cluster across availability zones, so the identity service tolerates $N-1$ broker failures without an outage — the same posture etcd or CockroachDB take, applied to secrets.
- **VM-side SDK in Python (`osvault`)** for the bootstrap flow — handles attestation, token refresh, leasing, and audit. The broker itself is Go.

2.2.2 Bare-metal datacenter lifecycle

My current piece of work at Juspay is on `namma-cli`, a Rust CLI that drives the lifecycle of bare-metal hosts in our datacenter, sitting on top of MAAS for hardware control. Hosts are described declaratively in site specs; `namma-cli` reconciles spec to actual state. The architectural shape is similar to Cluster API, scoped to bare metal rather than VMs.

I am working on the `wipe-disk` command — secure-erase of all drives on a host before reprovisioning, using NVMe Sanitize where the controller advertises it and falling back to `hdparm --security-erase`. The CLI orchestrates: power-on via IPMI, MAAS commission into an ephemeral OS, push the wipe script over SSH, run it, wait for `poweroff -f`, MAAS releases the machine. End-to-end across a real bare-metal pipeline.

2.3 Independent project: PostgreSQL on Kubernetes / OpenStack

A 5-layer nested virtualization stack on a single i9-14900 box: bare-metal Linux at the bottom, KVM running an OpenStack tenancy on top, Kubernetes running inside that, CloudNativePG inside Kubernetes, Postgres inside CNPG. The work was equal parts performance tuning, lifecycle automation, and identity plumbing.

Performance tuning. The inner-VM Postgres saw write-TPS climb roughly 50% after two pieces of work — NUMA-aware CPU pinning of the Postgres workers to the same socket as the L1 hypervisor exposing the disks, and 16 GB of hugepages dedicated to the inner VM. The buffer pool was thrashing the TLB at default page sizes; 2 MB pages cut TLB misses enough that the syscall cost on every page fault went away.

Lifecycle automation. Cluster API for OpenStack (CAPO) provisions VMs declaratively; CloudNativePG is the Kubernetes operator that runs the Postgres cluster. End-to-end, a single `kubectl apply` brings up an HA Postgres cluster (one primary + N replicas, synchronous WAL streaming) in ~12 minutes from cold. Failover is automatic — CNPG runs a leader-election loop, demotes the failing primary, promotes a healthy replica, and rewires the service endpoint. Tested under primary-VM kill and primary-VM network partition; recovery to write-ready in roughly 7 seconds with RPO=0 (no acknowledged write lost, because the synchronous stream guarantees the standby has it before the primary acks the client).

Identity plumbing. A Go-based OpenBao broker plus the `osvault` Python SDK on the VM side delivers secrets at bootstrap — the VM proves its OpenStack-assigned identity via IMDS, the broker validates it, the SDK fetches the secrets it is allowed to.

2.4 urunc so far — what I have learned through issue #389

A few weeks ago I picked up [urunc-dev/urunc#389](#) — high host CPU caused by console output in running containers. A prior contributor had attributed the regression to `bufio.Scanner` in `cmd/urunc/log_forward.go`. My own profiling showed that path is not hot during guest execution: that scanner only handles the parent-child JSON log pipe during `urunc create`; once `syscall.Exec()` runs the VMM manages stdio directly. The real cost lives one layer deeper.

I wrote a small KVM-exit and throughput sampler — `bench-console.sh`, which @ananos has since indicated they would like to fold into CI as a regression check — and traced the regression to per-byte PIO traps on the 16550 UART at port `0x3F8`. The `-serial stdio` baseline costs roughly 1.16 M PIO exits per 5-second window, with QEMU's `address_space_write` dominating the host flame graph. A simple `virtio-console` swap drops PIO exits to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×), closing the runc-vs-urunc gap on this workload from roughly 407× down to ~11×.

The thread has since covered: how to differentiate `virtio-console` from `serial` without breaking Linux kernels built without `CONFIG_VIRTIO_CONSOLE` (my first proposal, a capability-intersection design borrowed from the rootfs path, was constructively rejected by @cmainas for exactly that reason); the upcoming pivot from the `com.urunc.unikernel.cmdline` annotation to a `kernelArgs`-style annotation that lets users specify boot parameters at build time; and an invitation from @ananos to draft a blog post on the investigation for the CNCF blog.

Through that work I have gained working familiarity with:

- Monitor invocation path — `pkg/unikontainers/hypervisors/qemu.go`, `BuildExecCmd`, and the `syscall.Exec` -style handoff in `unikontainers.go`.
- The distinction between the parent-child JSON log pipe (consumed by `bufio.Scanner` before `syscall.Exec`) and the VMM-managed stdio path that takes over after.
- The QEMU + KVM monitor surface — what costs PIO exits at the 16550 UART, how virtio devices change the picture, where the cost actually lives.
- Linux unikernel path — `pkg/unikontainers/unikernels/linux.go`, the boot-params construction, the urunit handshake.
- The `VMM` interface in `pkg/unikontainers/types/types.go` and how each monitor (QEMU, Firecracker, HVT) implements it today.
- An empirical debugging posture — `/sys/kernel/debug/kvm/io_exits`, `perf` flame graphs, and a small reproducer image, rather than guessing.

3. The project — monitor lifecycle for urunc#112

3.1 The problem

urunc today shells out to QEMU or Firecracker with a long command string. That is enough to *start* a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to shut down gracefully, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process. Once `syscall.Exec` hands off to the monitor at `unikontainers.go:533`, the only handle urunc retains is the PID. Both QEMU and Firecracker already expose proper socket APIs — QMP for QEMU, a REST-style API socket for Firecracker — and the [slide deck for #112](#) calls out a secondary benefit: the `reexec` process currently blocks on `AwaitMsg(StartExecve)` between the create and start phases, and that gap can be used for parallel VMM init for a ~30 ms boot win.

3.2 Driving each monitor through its API end-to-end

Before writing the proposal I wanted to see what QMP and the Firecracker API actually look like in practice, not just in the docs. urunc itself I have been running locally since I filed #389 — standard build install, a busybox-on-Linux container as the baseline workload for the bench scripts — but the monitor APIs were the part I had not directly touched, so I wrote two small Go clients and drove each monitor through a real lifecycle.

The QMP run starts `qemu-system-x86_64 8.2.2` paused with `-S -qmp unix:<sock>,server,nowait` and then walks through the greeting, `qmp_capabilities`, `query-status`, `cont`, `query-cpus-fast`, `stop`, `quit`. The whole exchange is around 2 ms on a local Unix socket. The non-obvious thing I learned: QMP interleaves async events with command replies on the same socket. My first version of the client was naive — one read per command — and it worked through the handshake, then `cont`'s `RESUME` event arrived before the `cont` reply, every read after that was off-by-one, and `query-cpus-fast`'s reply turned up where I was expecting `stop`'s. The fix is to type-dispatch every frame: `{return}` or `{error}` is a reply to the most recent in-flight command; `{event, timestamp, data}`

is async telemetry that goes to a separate subscriber. It is the kind of thing that does not surface from reading docs.

The Firecracker run is more involved because Firecracker has no CLI for guest config — only `--api-sock`. Background mode is the only mode it has. I drove the full lifecycle through the API: `GET /` to read the instance info, `PUT /machine-config` with 1 vCPU and 128 MiB, `PUT /boot-source` with the kernel path and cmdline, `PUT /drives/rootfs` with an Ubuntu 22.04 ext4 image, `PUT /actions {InstanceStart}` to boot, `GET /` to verify the state moved to `Running`, `PATCH /vm {Paused}` and `{Resumed}` to exercise pause and resume, and finally `PUT /actions {SendCtrlAltDel}`. The microVM booted, reached systemd, honored the Ctrl-Alt-Del, and Firecracker exited with `exit_code=0`. The only call that took meaningful time was `InstanceStart` at 16 ms — the actual KVM boot. Everything else was 0.3–0.5 ms. Three things contrast with QEMU and matter for the interface design: FC has no CLI for guest config, the API socket is strictly request/reply (no events on the control channel — though `/logger` and `/metrics` provide event-like streams separately), and there is no `device_add` equivalent — you can `PATCH` existing drives but cannot attach a new device after boot.

After both wire flows checked out I wanted to verify the same thing against a real running Linux guest, not just a paused dummy. I started QEMU with the same `-S -qmp` setup plus an Ubuntu cloud image attached, sent `cont`, waited for the guest to boot (Ubuntu 5.15 came up cleanly through SeaBIOS and iPXE), then exercised the lifecycle ops on the running guest: `stop` and `cont` to pause and resume, `query-block` to read the attached qcow2 path, `query-cpus-fast` for vCPU thread IDs, `netdev_add` and `device_add` to hotplug a virtio-net-pci on the fly, `query-pci` to confirm it appeared, `device_del` to unplug, and finally `quit` for a clean shutdown.

A machine-type subtlety surfaced from this last run. I had explicitly started QEMU with `-machine q35` in my test script, and the first hotplug attempt failed with `Bus 'pcie.0' does not support hotplugging`. On q35 the root PCIe bus exposes itself as `pcie.0`, which is not hotplug-capable; you have to pre-allocate one or more `pcie-root-port` devices at startup and target them via `bus=rp1`. I added `-device pcie-root-port,id=rp1,chassis=1,slot=1` and the hotplug worked. The `device_add` itself took ~11 ms — QEMU device realization (BAR allocation, MSI-X table setup) plus the guest's hotplug interrupt and driver probe. urunc's `qemu.go` currently does not pass `-M` so it uses QEMU's x86_64 default (`pc-i440fx`), where hotplug works on the legacy `pci.0` directly — so this is not a current urunc bug, but the machine-type choice becomes a real design-doc decision the moment hotplug enters the runtime.

The transcripts and run scripts are in this repo under `prototypes/` for anyone who wants to reproduce the runs.

3.3 My approach — invert the timing, keep everything else

The smallest change to urunc's existing flow that achieves what #112 wants is to invert the timing of the existing `execve` and have the monitor start *paused*. Concretely, in `unikontainers.go:533` the reexec process currently calls `syscall.Exec` only after `urunc start` has signalled `StartExecve`; the monitor then immediately runs the guest. Move the `execve` earlier — right after `CreateContainer` hooks — and have `BuildExecCmd` produce a paused command (QEMU's `-S -qmp unix:<sock>,server,nowait`, Firecracker's `--api-sock <sock>` with no boot action yet). Reexec is

gone the moment `execve` returns. The monitor process is now alive, paused, listening on its API socket. `urunc start` then connects to that socket and sends the "start guest" command (`cont` for QMP, `PUT /actions {InstanceStart}` for FC), verifies the state, runs `Poststart` hooks, and exits.

What this preserves:

- `containerPid == monitor PID` . Same as today. The shim, containerd, the OCI hooks, the pid-file — none of them see a different process model.
- **The `execve` handoff and the namespace inheritance.** CreateContainer hooks still run in `reexec` before `execve` ; the monitor inherits the same namespaces from the same call. No new code path.
- **The `VMM` interface in `pkg/unikontainers/hypervisors/types.go` .** Same shape, extended with the post-spawn operations `urunc` needs — probe state, pause, resume, shutdown, hotplug net/block, attach console — and a capability flag for backends that lack a socket (HVT keeps the current CLI-only path).

What this changes:

- **The existing `urunc` IPC handshake becomes vestigial.** `StartExecve` and `StartSuccess` on `urunc.sock` exist today to coordinate `reexec` and `urunc start` across the gap. With the monitor paused on its own socket, the verification `urunc start` needs is "is the monitor socket responsive?" — which is a stronger check than the current "we assume the `execve` succeeded." The IPC sockets can be removed cleanly.
- **The parallel-init opportunity falls out for free.** Between `reexec`'s `execve` and `urunc start`'s `cont` , the monitor is doing device emulation setup, kernel-image loading, virtio queues — exactly the work the [slide deck](#) targets. The Firecracker probe puts a number on it: only `PUT /actions {InstanceStart}` costs real time (16 ms); every config call is sub-millisecond, and all of them happen before the start command lands.
- **Lifecycle ops after start (`urunc kill` , `urunc delete` , future `urunc state` or `urunc attach`) connect fresh to the persisted socket, send the operation, close.** Per the QMP probe, that round-trip is sub-millisecond. Fallback when the socket is dead is signal-to-PID, exactly as today.

Both prototypes in §3.2 already validate this exact flow on the wire side: the QMP probe starts QEMU with `-S` and sends `cont` ; the Firecracker probe starts FC with `--api-sock` only and sends `PUT /actions {InstanceStart}` via the API. The `urunc`-side code change — moving where in `reexec` the `execve` happens, removing the `urunc` IPC handshake, teaching `urunc start` to talk to the monitor socket — is real work, but it is well-scoped and doesn't introduce any new process-lifetime concepts.

The alternative I considered was a long-running supervisor process that owns the monitor and the persistent socket connection. I think that is the wrong direction here. It changes `containerPid` semantics (the supervisor becomes the container's init process, not the monitor itself), introduces a new zombie state (live supervisor + dead monitor), and the persistent-connection benefit it provides is small — Unix-socket reconnect is sub-millisecond per the QMP probe, and async-event subscription is the only thing that genuinely benefits, which can be a separate event-sink process rather than the container's init. The invert-the-timing approach above does more with less.

3.4 Concrete changes in code

- `pkg/unikontainers/hypervisors/qemu.go` — `BuildExecCmd` adds `-S` `-qmp` `unix:<sock>,server,nowait`. If the design doc keeps the current i440fx default, hotplug works on the legacy `pci.0` directly; if it moves to q35 for PCIe-native reasons, `BuildExecCmd` would also need to pre-allocate a bank of `pcie-root-port` devices at startup — typically four to eight, since each root port is a single slot — and `device_add` would target them via `bus=rp1`, `bus=rp2`, etc. Implement the new control methods for QMP: `query-status` (probe), `quit` (replaces the current `killProcess` in `Stop`), `cont / stop` (pause/resume), `device_add / device_del` (hotplug for virtio-net and virtio-blk first), `chardev-add / chardev-remove` (console attach/detach). Wire types hand-rolled from the prototype; OOB priority on `query-status` and `quit` would let us reach a wedged guest whose main command queue has stalled.
- `pkg/unikontainers/hypervisors/firecracker.go` — `BuildExecCmd` moves to `--api-sock <sock>` -only. The existing per-VM config (machine-config, boot-source, drives, network) moves out of `BuildExecCmd` and into a "configure" step that runs against the API socket between `execve` and `urunc start`. Implement the same control methods via `firecracker-go-sdk`, with capability flags marking the differences (no `device_add`, no async events).
- `pkg/unikontainers/hypervisors/types.go` — extend the `VMM` interface with the new control methods plus a capability-flag method (`SupportsMonitorControl()` or similar, in the same shape as the existing `SupportsSharedfs`). HVT and any future no-socket monitor return false and keep the current CLI-only path.
- `pkg/unikontainers/unikontainers.go` — `Exec` (currently ending at `syscall.Exec` on line 533) now builds the paused command and execves into the monitor without waiting for `StartExecve`. The `SendMessage(StartSuccess)` at line 519 is removed; verification moves to `urunc start`'s socket-connect.
- `cmd/urunc/create.go` — the reexec branch (line 347 onward) no longer creates the `urunc.sock` listener or waits on `AwaitMsg(StartExecve)`. After `CreateContainer` hooks it falls straight through to `unikontainer.Exec`.
- `cmd/urunc/start.go` — instead of sending `StartExecve` via the `urunc` IPC, opens a connection to the monitor's API socket (path read from `state.json`), sends the configure-and-start sequence (`cont` for QEMU; the FC config `PUT` s followed by `InstanceStart` for FC), verifies state, runs `Poststart`. The connect uses the same retry-with-backoff pattern `sendIPCMessageWithRetry` in `pkg/unikontainers/ipc.go` already uses for `urunc`'s own sockets — the monitor may not have its socket up immediately after `execve`.
- `cmd/urunc/kill.go`, `cmd/urunc/delete.go` — attempt graceful shutdown via the API socket (QMP `quit`, FC `SendCtrlAltDel`) first; fall back to signal-to-PID when the socket is unresponsive.
- `pkg/unikontainers/ipc.go` — for socket-capable monitors the `StartExecve / StartSuccess / StartErr` messages and the `urunc.sock / reexec.sock` plumbing become unused. They cannot be fully removed though: HVT (and any future no-socket monitor) keeps the legacy gate, so `ipc.go` stays as the fallback path that the capability flag in `types.go` routes around. The `sendIPCMessageWithRetry` helper itself is still useful — `urunc start` uses it for the monitor-socket connect too.

3.5 Open questions the design doc will resolve

A handful of things I have not settled, and would not pretend to settle in two weeks of reading. These are the questions I would expect the design-doc PR in weeks 1–2 to spend most of its time on, and I am flagging them here so it is clear which parts of §3.3 and §3.4 are positions I am confident in versus working hypotheses.

- **Where the monitor socket path lives.** The obvious answer is `/run/urunc/<id>/qmp.sock` or `fc.sock` inside the existing per-container state directory, with the path written into `state.json` at create time so `urunc start` can find it on a fresh exec. But stuffing `urunc`-internal paths into the OCI `state.json` is a choice that affects shim integration, and I would want to settle it explicitly rather than smuggle it in.
- **Monitor death between `execve` and `cont`.** In the new flow the reexec process is gone the moment `execve` returns. If the monitor crashes during its own init — after `execve` succeeds but before `urunc start` connects — nobody in `urunc`-land is alive to notice. The natural answer is that `urunc start`'s connect-with-retry simply times out and reports container-failed, but the error path needs to be written down rather than assumed.
- **Socket-readiness race.** Between the monitor's `execve` and its `listen()` call there is a small window where the socket path does not exist yet. Retry-with-backoff (the existing `sendIPCMessagesWithRetry` shape) papers over it for short delays; a more robust option is to open the listener in reexec before `execve` and pass the fd to the monitor via the numbered file descriptor convention QEMU and FC both support. Design-doc choice — both work.
- **Firecracker configure-step ownership.** FC's per-VM configuration (`machine-config`, `boot-source`, drives, network) can run either inside reexec right after the `execve` — which gives the parallel-init flavor the slide deck targets — or inside `urunc start` serially before the final `InstanceStart`. Both produce a correct VM; the latency profiles differ, and the failure-mode story differs too. Pick one in the design doc.
- **Machine type and hotplug capacity.** Stay on i440fx (where hotplug works on `pci.0` directly) or move to q35 (PCIe-native, needs a bank of pre-allocated `pcie-root-port` devices, more modern). I do not have a strong opinion here yet — i440fx is simpler and is what `urunc` uses today; q35 is the direction modern QEMU is moving. Worth a paragraph in the design doc, not a unilateral choice.

3.6 Out of scope and what I will verify

Out of scope for this LFX even if time allows: rewriting `urunc`'s seccomp handling (`PreExec` for HVT), adding a third monitor (cloud-hypervisor, Dragonball — the interface should make it easier, but doing it is somebody else's first PR), introducing a heavy QMP or FC client library when the prototypes already show both wires are small enough to hand-roll, and rewriting the existing reexec/`urunc`-start IPC channels.

Before each merge I commit to verifying: the QMP-controlled QEMU path can spawn paused, query state, gracefully shut down, and hotplug a virtio-net device end-to-end with the existing nginx-on-unikraft integration tests staying green; the Firecracker path can do the equivalent subset; the CLI-only fallback continues to work for HVT and any monitor that does not expose a socket; an extended `bench-console.sh` measures per-round-trip cost on both backends and lands in CI as a regression check.

4. Plan — 12 weeks (8 Jun – 31 Aug 2026)

The design doc opens as a draft PR from week 1 and stays open through week 3 for asynchronous comments. Implementation is broken into multiple small PRs so review happens in parallel with development. The verification work in §3.2 reduces risk on weeks 4–7 — I am not learning QMP or the Firecracker API for the first time during the implementation.

WK	PHASE	DELIVERABLE
1–2	Onboarding + design doc draft	Re-read <code>pkg/unikontainers/hypervisors/</code> with mentor context. Open a draft design-doc PR covering the <code>MonitorControl</code> interface, the QMP and Firecracker mapping, the reexec flow change, socket lifecycle, and the opt-in vs default-on policy.
3	Design doc sign-off	Address review on the draft PR. Interface signature and dependency decisions settled before any production code lands.
4–5	QEMU + QMP integration	Extend the qmp-probe code into urunc's QEMU backend. <code>BuildExecCmd</code> produces a paused command. <code>urunc start</code> drives <code>cont</code> via the QMP socket. The reexec flow change (<code>execve</code> moves earlier, urunc IPC handshake removed) lands here behind a feature flag. The legacy path stays the default until week 8.
6–7	Firecracker + lifecycle ops	Same flow for FC via <code>firecracker-go-sdk</code> : paused <code>--api-sock</code> startup, configure-then- <code>InstanceStart</code> from <code>urunc start</code> . Implement the lifecycle ops on both backends — <code>quit</code> / <code>SendCtrlAltDel</code> in <code>urunc kill</code> / <code>delete</code> , pause/resume, hotplug (QEMU only — FC's capability flag is false).
8–9	Tests, CI, hotplug demo	Integration tests in the existing layout. Extend <code>bench-console.sh</code> to time the new round-trip costs. End-to-end demo: start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity inside the guest.
10	Docs + bug-fix	Review-comment cleanup across all open PRs. User docs covering the QMP socket and how to opt into the new control path.
11–12	Buffer + blog post	Reserved for slippage. If none, take a follow-up issue from the maintainers' backlog. Mentee blog post for the CNCF / Nubificus blog covering the redesign and the numbers.

The riskiest piece is the timing change at `unikontainers.go:533` — moving the `execve` earlier and removing the urunc IPC handshake. Mitigation: feature-flag it from week 4; the legacy "execve-on-StartExecve" path stays default until the new path has green integration tests in week 8. The wire-protocol risk on QMP and the Firecracker API is low: both prototypes in §3.2 already exercise the exact handshakes the production code will use.